From Description to Interactive VR Scene: An LLM Agent for Authoring Immersive Experiences

Rongqian Chen[1] Allison Andreyev[1] Xuan Wang[2] Jianan Jiang[2] Bin Li[2] Tian Lan[1]

[1]George Washington University [2]Pennsylvania State University

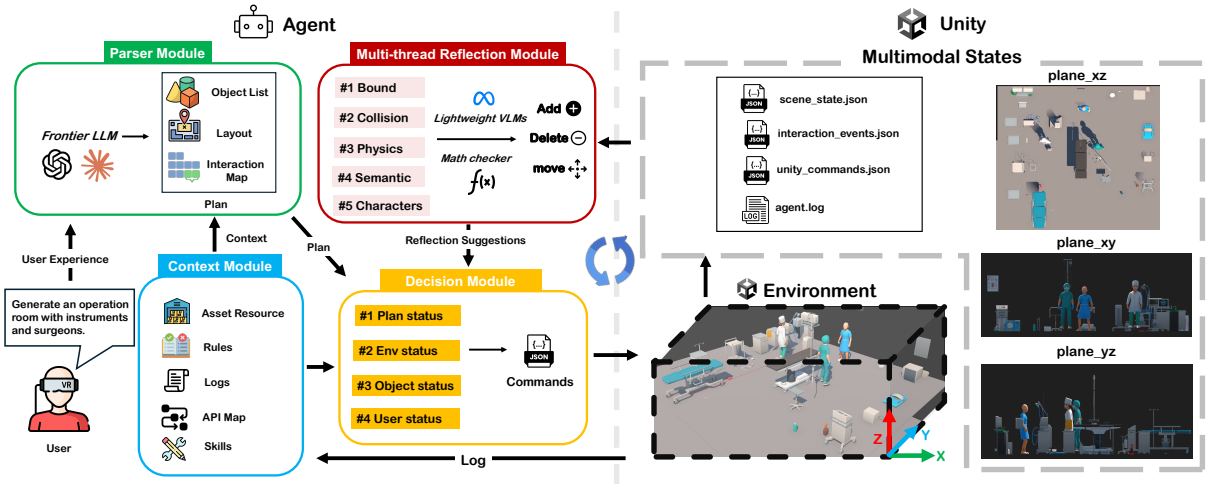


Figure 1: Environment generation pipeline. A frontier-LLM Parser turns a user experience description, grounded by the Context module (assets, rules, logs, skills), into a structured plan. Then, a Multi-Thread Reflection module runs five lightweight-VLM and rule-based threads (Bound, Collision, Physics, Semantic, Character) over three orthogonal screenshots and the live scene state, emitting typed corrections. The Decision module fuses plan, scene, object, and user status into Unity commands; Unity returns the multimodal artifacts that close the loop.

Abstract

Authoring interactive immersive scenes for healthcare training and patient education today requires sustained collaboration between developers, 3D-modeling artists, and clinicians, putting immersive medical visualization out of reach for most domain experts. We present an LLM-based multi-module agent that turns a single natural-language description into a fully populated, interactive VR scene, lowering the authoring barrier from code and 3D tooling to natural language commands. A closed loop between a Python reasoning core and a Unity runtime combines a frontier-LLM planner with lightweight VLM critics grounded in three orthogonal screenshots. A separate interaction layer compiles affordances and story logic into a JSON so runtime response is LLM-free. Across fifteen diverse healthcare scenarios (45 trials), the agent recovers the correct object set in every scene at 5.1 frontier-LLM calls and 113 s per trial, and generates coherent multi-beat story lines averaging 5.0 beats per scenario.

Index Terms: Large language models, healthcare visualization, interactive immersive experience, AI agent, procedural generation.

1 Introduction

Interactive immersive experiences empowered by Virtual Reality (VR) have demonstrated potential in providing safe, realistic, and repeatable training for complex medical scenarios such as mental health interventions [4] and clinical skills training [17]. However, developing such experiences typically demands sustained collaboration between software developers, 3D-modeling artists, and healthcare professionals, resulting in high costs and long development cycles that limit scalability and personalization.

Recent advances in large language models (LLMs) have enabled a new class of autonomous agents capable of complex reasoning, planning, and multi-step interaction across diverse domains [20, 22]. In healthcare specifically, agentic AI systems have begun to operate autonomously toward clinical goals [2, 10], spanning evidence-based diagnosis [6, 21] and tool-augmented medical reasoning. These efforts, however, focus on analytical clinical tasks rather than on driving interactive user experience. Automatically translating natural language scene descriptions into immersive healthcare scenarios with automated interactions as VR systems remains an open problem.

In this paper, we present an LLM-based multi-module agent designed to generate interactive healthcare experiences from plain English descriptions. A clinician or experience designer can describe a desired user experience (for example, "guide a post-surgery patient through a rehabilitation exercise sequence with adaptive difficulty"), and the agent automatically generates the corresponding environment, interactions, responses, and feedback mechanisms. Inspired by cognitive agent architectures [14], our system is organized as a closed-loop pipeline of five modules (Parser, Context, Multi-Thread Reflection, Decision, and a Unity-side Operation layer) that exchange a structured plan, a live scene state, and three orthogonal screenshots. Reflection is parallelized across bound, collision, physics, semantic and Character threads, each backed by a deterministic checker, allowing the agent to handle complex healthcare scenarios while maintaining geometric soundness.

Our contributions are as follows:

- A multi-module agent architecture that lowers the authoring barrier for immersive medical visualization, generating complex VR scenes and per-object interaction logic from natural language user experience descriptions alone.
- A hierarchical planning mechanism that decomposes high-level instructions into structured plans and executable actions, with closed-loop reflection for adaptive behavior.
- An evaluation across fifteen specialty-diverse healthcare-VR scenarios (emergency, ICU, surgical, dental, ophthalmology, radiology, maternity, lab, cosmetic) showing that natural-language authoring reliably recovers the required object set and produces multi-beat interaction story lines.

2 Related Work

Immersive Healthcare Applications. Virtual and augmented reality have several applications in healthcare, from acute pain distraction [7] and treatment of mental health disorders [4] to mixed-reality trauma simulation for medical skills training [17]. More recent systems embed AI agents in hand-authored clinical VR content [1, 13, 24], but none automatically generate complete interactive experiences from high-level descriptions. While these systems integrate AI agents into specific, hand-authored clinical use cases, none automatically generate complete interactive experiences, including both the scene and its interaction logic, from high-level descriptions.

Language-Driven 3D Scene Generation. A parallel line of work generates or edits 3D scenes from natural language. LayoutGPT [3] prompts an LLM to emit object layouts for indoor scenes, and Holodeck [23] extends this to full embodied-AI environments by combining LLM-proposed spatial constraints with an optimization step that places retrieved assets. LayoutVLM [15] instead uses a vision-language model to produce layout objectives that are refined by differentiable optimization. Several systems implement closed-loop visual feedback: SceneCraft [8] has an LLM agent write Blender code and revises placement using rendered images of the scene, and Chat2Layout [18] supports interactive layout refinement with a multimodal LLM. Our reflection loop is similar, however instead of static scenes we generate an interactive experience with story beats, object interactions, and narrative sequencing for human learners, and we operate under a fixed iteration budget to keep the agent lightweight.

LLM-Based Agents. LLMs have given rise to autonomous agents that can decompose tasks, reason about state, and execute multi-step interactions [20, 22]. Park et. al [11] demonstrate that LLM-driven agents can simulate believable human behavior in interactive worlds. In game-like and computer-use settings, frameworks such as CRADLE [16] and Voyager [19] show that modular architectures combining perception, planning, action, and reflection, in line with broader cognitive architectures for language agents [14], can navigate and interact with complex virtual environments. In the medical domain, recent agent systems target clinical diagnosis [6, 21] or simulate hospital workflows with role-playing LLM agents that train each other on synthetic patient cases [9], but these systems operate over text or image-based clinical reasoning rather than orchestrating real-time interactions in spatial, immersive environments. Our work adapts modular agent design to the latter setting, where the agent must orchestrate real-time interactions within a spatially and semantically valid scene.

3 Method

3.1 System Architecture

Our agent operates as a closed-loop pipeline that couples a Python reasoning core with a Unity simulation environment (Figure 1). Given a free-form user experience description, the agent iteratively plans, executes, reflects, and revises until the scene satisfies bounded, collision-free, physically plausible, and semantically coherent constraints. A central design choice is that the pipeline is built around two LLM tiers with complementary strengths. Plan quality determines eventual scene quality—an incorrect object list or layout intent cannot be repaired by any amount of downstream reflection. Therefore, we commit a frontier LLM to a small number of high-leverage calls, namely scene synthesis, interaction-map generation, and story-line generation. In contrast, reflection and per-step decision making are high-frequency operations whose value depends on iteration speed, not single-shot reasoning depth: each plan step may trigger several VLM rechecks and command repairs. We therefore route these calls to a fast open-weight model, which allows the agent to close the feedback loop in seconds.

Parser Module. A frontier LLM ingests the user's experience description together with context drawn from the Context module's asset catalog and authoring rules. It outputs a structured plan comprising three artifacts: an object list (typed prefab references with role tags), a layout (per-object 3D anchor positions, surface placement hints, and clearance budgets), and an interaction map (event-driven bindings between user triggers and animation/state transitions). The plan is persisted as `plan.json` and consumed by the downstream modules.

Context Module. The Context module functions as the agent's long-term memory and tool index. It stores the asset resource (available prefabs and their categories), authoring rules (room-type templates, surface keyword maps, character role conventions), execution logs (a rolling history of past commands and outcomes), an action map (the static mapping from high-level operators to Unity API calls), and reusable skills (parameterized animation and lighting routines). The module shares its contents to downstream modules through two complementary access paths: i) small, schema-bound resources such as the action map and the active skill set—injected directly into the planner—and ii) decision prompts—free-form authoring rules and execution logs are retrieved on demand from a vector store keyed on the current plan step.

Multi-Thread Reflection Module. Rather than rely on a single critic, we parallelize reflection across five specialized threads, each pairing a lightweight VLM with a deterministic checker so that every LLM verdict is corroborated numerically. (i) 'Bound' clamps placements to the room's $[x_{\min}, x_{\max}] \times [z_{\min}, z_{\max}]$ envelope and reverifies them under the VLM. (ii) 'Collision' runs Axis-Aligned Bounding Box (AABB) pair tests for geometric overlap and a VLM recheck on three orthogonal screenshots (xy, xz, yz). (iii) 'Physics' flags floating or unsupported objects whose vertical position is inconsistent with any plausible parent surface beneath them. (iv) 'Semantic' verifies support and containment relationships (e.g., small props rest on parent surfaces with $y \geq 0.7$) using both height tests and a VLM coherence judgment over the same screenshots. (v) 'Characters' ensures that every human role parsed from the experience description is present in the scene, adding any characters that earlier passes dropped. Each thread emits typed correction lines (`add`, `delete`, or `move`) over the live `scene_state.json`; lines are sanitized against the canonical name set and merged into a single reflection suggestion record.

Decision Module. The Decision module fuses four status views to produce executable commands: plan status (which step is in progress and how many reflection rounds it has consumed),environment status (the latest scene state and screenshots), object status (placed objects, their parents, and any flagged conflicts), and user status (interaction triggers and the current user position). Conditioned on these, an LLM call synthesizes a batch of typed commands (`PlaceObject`, `PlaceHuman`, `MoveObject`, `DeleteObject`, animation and lighting controls) that are validated against the action map, repaired if syntactically incomplete, and dispatched to Unity.

Unity Operation and Multimodal States. The Unity component executes incoming commands, advances the sim-

ulation, and writes back four artifacts that close the loop: `scene_state.json` (authoritative object inventory with poses), `interaction_events.json` (user-triggered events resolved by the runtime), `unity_commands.json` (the executed command record), and `agent.log` (operator-level diagnostics), together with three orthogonal screenshots that grant the Reflection module a near-complete spatial view without requiring a heavyweight 3D scene encoder. These artifacts feed the next iteration's Reflection and Decision passes, terminating when the plan is complete and no thread issues further corrections.

3.2 Interaction

The interaction layer consists of two complementary artifacts, both produced up front by the frontier LLM and persisted as a JSON (Figure 2).

Interaction map (environment-inherent). The first artifact specifies the interactions inherent to the placed objects, independent of any task. Rows enumerate entities currently in the scene (user, doctor, monitor, instrument table, MRI scanner, etc.) and columns enumerate a small fixed set of typed action primitives (gaze, grab, release, touch, and etc.). A populated cell records the deterministic response the runtime emits when that action is performed on that entity (a dialogue line, an animation, a state transition). Because the map is keyed on role tags rather than on prefab identity, the same prefab can expose different affordances across scenarios; a doctor entity responds to gaze and touch with dialogue, whereas a supply trolley responds to grab and release with highlights.

Story line (task-conditioned, sequential). The second artifact decomposes the user's high-level request into an ordered chain of beats, where each beat is the smallest narrative unit the runtime can advance. A beat binds a target object, the required user action that triggers it (drawn from the same primitive vocabulary as the interaction map), and a response payload that is itself a sequence of one or more Unity actions, so a single beat can chain multiple effects (e.g., a dialogue line followed by a walk-up animation) into one trigger event. Beats are strictly sequential: only the current beat's trigger is active, and the next beat unlocks only after the current trigger fires. At runtime, every user input is matched twice in parallel: once against the interaction map for the per-object affordance response (which fires regardless of task progress), and once against the active beat's (object, action) pair to drive story progression. For "Bring the supply trolley to the surgeon," the chain is (i) gaze at the doctor, (ii) grab the supply trolley, (iii) release at the surgeon; resolved events are appended to `interaction_events.json` for the next planning iteration to consume.

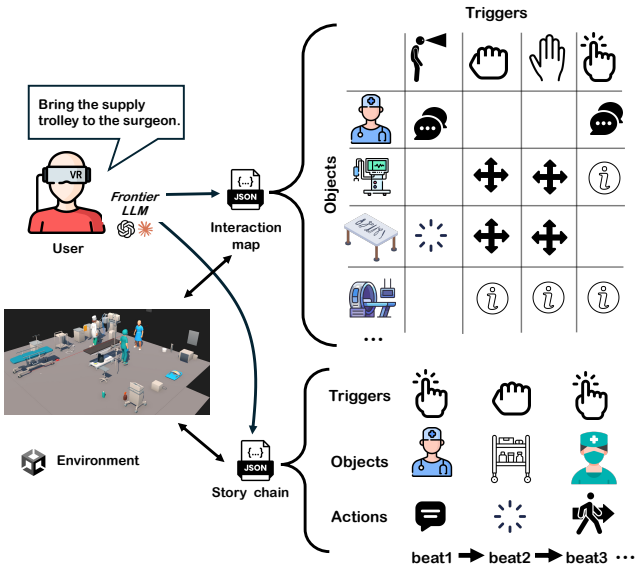


Figure 2: Interaction pipeline. The frontier LLM emits two JSON artifacts up front: an *Interaction map* of per-object action affordances over a small set of typed primitives (gaze, grab, release, touch), and a *Story line* of strictly sequential trigger steps, each binding a target object, the required user action, and the response the runtime emits on success. After generation, the Unity runtime services user input against both artifacts without further LLM calls, enabling real-time feedback.

4 Experiments

Scenarios. We evaluate the pipeline on fifteen healthcare-VR scenarios (S01-S15) spanning specialty diversity. Together they assess the pipeline along seven axes: room size (from a 3×3 m supply closet to an 8×8 m nurse station), object density, scene complexity, character count, spatial-constraint type, semantic specificity, and prefab-category coverage. All scenarios draw from a shared 3D asset library, the commercial *ithappy Medical Pack* on the Unity Asset Store, which supplies 440 low-poly medical-equipment prefabs and 24 character prefabs with Mixamo-compatible animations. Each scenario is supplied as a single narrative `experience_S{NN}.txt`: a free-form scene description and an `[interactions]` block in plain English. Note that the agent receives no object list, no anchor coordinates, and no room-config block; these are inferred at trial start by a frontier-LLM scene-synthesis pass.

Pipeline and protocol. Each trial uses a two-tier LLM stack. A frontier model (default `gpt-5.4-mini`) is invoked: scene synthesis (narrative to `cfg+plan`), interaction-map generation, and story-line generation. And another frontier model (claude-sonnet-4.5) is used for LLM-as-judge. A lightweight model (`Llama-4-Scout-17B` via Groq) drives the high-frequency placement loop and the three VLM-backed reflection threads (Bound, Collision, Semantic), with the 'Physics' and 'Character' threads running deterministically. While it is plausible that additional iterations may improve the end result, we implement a three-iteration cap for reflections per plan step in order to minimize latency and preserve our main contribution of a lightweight agent. Robustness layers retry frontier JSON on parse failure, re-send unacknowledged `ClearScene` commands to Unity, and run a 'Liveness Check' that aborts vacuous trials before they pollute the aggregate. Each scenario is run for three independent trials with different seeds and decoding temperatures; results are reported as means across trials.

Phase-1 environment generation metrics. For environment generation, we compute deterministic metrics from the JSON artifacts emitted by the Operation layer, plus two LLM-as-judge ratings. Collision rate is the fraction of pair overlaps over placed bounding boxes; step-completion rate is the fraction of plan steps the reflection log marks closed within the three-iteration cap. Cost is captured by the frontier-LLM call count and Phase-1 wall-clock seconds per trial. The two LLM-as-judge ratings on a 1-5 scale, coherence and spatial, are produced by claude-sonnet-4.5 over the experience description and the three orthogonal screenshots, triangulating object-selection quality and layout reasoning, respectively.

Phase-2 interaction metrics. For the interaction layer, we compute static coverage and chain-richness metrics over the generated interaction map and storyline, plus two LLM-as-judge ratings on the generated narrative content. Ambient-object coverage is the fraction of placed objects with at least one interaction binding. Chain length is the mean number of beats in the generated storyline, and actions per beat is the mean number of Unity actions a beat dispatches when fired. The two LLM-as-judge ratings on a 1-5 scale, object informativeness and narrative coherence, evaluate whether per-object dialogue is specific and useful, and whether the storyline follows a setup, action, and payoff structure.

Table 1: Phase-1 environment-generation results: per-scenario means across $n=3$ trials each (45 trials total). *Coll* = collision rate (%); *Plan* = step-completion rate (%); *LLM* = mean frontier-LLM call count per trial; *Time* = mean Phase-1 wall-clock seconds per trial; *Coh*, *Spat* = LLM-as-judge coherence and spatial-reasoning scores on a 1–5 scale.

Scenario	Coll	Plan	LLM	Time	Coh	Spat
S01 ED Triage	5.8	100.0	4.7	159	5.00	3.00
S02 Hospital Ward	2.2	92.0	5.0	102	3.00	2.33
S03 Therapy Gym	2.9	100.0	7.7	146	4.33	3.67
S04 Surgery Prep	0.9	26.7	7.0	115	4.67	3.00
S05 Dental	1.4	77.8	6.0	107	4.00	2.67
S06 ICU	2.6	53.3	5.7	112	4.00	2.00
S07 Supply Closet	0.8	28.3	4.3	97	4.67	2.33
S08 Rehab Corridor	1.8	75.0	4.3	74	4.00	3.00
S09 Doctor's Office	2.9	80.0	5.7	94	3.33	2.33
S10 Nurse Station	2.8	80.0	5.3	111	4.33	3.00
S11 Radiology	9.3	86.7	4.0	91	4.00	2.00
S12 Maternity	7.6	90.5	3.7	119	4.67	3.33
S13 Ophthalmology	1.8	58.3	5.3	160	4.33	2.67
S14 Clinical Lab	3.9	80.0	4.3	107	4.00	3.00
S15 Cosmetic	2.2	80.0	4.3	99	4.00	2.33
Average	3.3	73.9	5.1	113	4.16	2.71

Table 2: Phase-2 interaction-layer results across the same 45 trials. *AmbCov* = ambient-object coverage (fraction of placed objects with at least one interaction binding); *Beats* = mean number of beats in the generated story line; *Acts/B* = mean number of Unity actions dispatched per beat; *ObjInf*, *Narr* = LLM-as-judge object-informativeness and narrative-coherence scores on a 1–5 scale.

Scenario	AmbCov	Beats	Acts/B	ObjInf	Narr
S01 ED Triage	1.00	4.0	5.33	3.33	3.67
S02 Hospital Ward	0.19	4.3	4.72	3.67	3.33
S03 Therapy Gym	1.00	5.0	4.13	4.00	3.33
S04 Surgery Prep	0.43	5.0	3.48	4.00	3.67
S05 Dental	0.65	5.0	2.83	4.00	3.33
S06 ICU	0.46	4.0	3.50	4.00	4.00
S07 Supply Closet	0.67	6.7	4.33	4.00	2.67
S08 Rehab Corridor	0.47	5.3	3.06	3.33	4.00
S09 Doctor's Office	0.62	5.3	2.64	4.00	4.00
S10 Nurse Station	0.44	5.3	3.56	4.00	4.00
S11 Radiology	0.75	5.0	3.27	4.33	4.00
S12 Maternity	0.56	5.0	3.12	4.33	4.00
S13 Ophthalmology	0.73	4.7	3.47	3.33	3.33
S14 Clinical Lab	0.74	5.0	2.00	4.00	3.67
S15 Cosmetic	0.94	5.7	3.13	3.67	4.00
Average	0.64	5.0	3.50	3.87	3.67

Phase-1 results. Table 1 reports per-scenario means for the environment-generation phase. Across all fifteen scenarios, frontier-LLM scene synthesis at trial start reliably recovers the object set from the narrative description alone, including the geometric extremes of S07's 3×3 m closet and S08's 3×10 m corridor. Plan-step convergence averages 73.9 %: open scenarios such as S01 and S03 close at 100 %, while the densest setups—S04 surgery prep at 26.7 % and S07 supply closet at 28.3 %—repeatedly hit the three-iteration reflection cap. Both scenarios pack many small props into a tight footprint, and the lightweight VLM critics struggle to localize these props in the orthogonal screenshots, so reflection cannot converge on a clean fix within the cap. Collisions remain low (mean 3.3 %, peaking at 9.3 % in S11, where large imaging machinery has tight clearance budgets). The cost profile is favorable: 5.1 LLM calls and 113 s of wall-clock per trial on average across the heavy-planner / light-critic stack. The LLM-judge ratings split sharply along the same axis: coherence averages 4.16/5, while spatial reasoning averages only 2.71/5—semantic object retrieval is typically the simplest task, and dense spatial layout, where current LLMs lack reliable spatial reasoning, is a bottleneck we identify for future work.

Phase-2 results. Table 2 reports the interaction layer. Story lines are substantively structured: chain length averages 5.0 beats per scenario (range 4.0–6.7) and each beat dispatches a mean of 3.5 Unity actions (range 2.0–5.3). Ambient-object coverage averages 0.64 (ceiling on S01/S03) and tracks how many placed objects the LLM judges salient enough to bind an interaction to. LLM-judged ratings on a 1–5 scale are 3.87/5 for object informativeness and 3.67/5 for narrative coherence; the only narrative score below 3.0 is S07 supply closet (2.67), the one scenario without characters and therefore without much narrative substrate.

Case studies. Figure 3 traces the agent through five replayed trials drawn from distinct specialties to illustrate how the quantitative metrics translate into concrete user experience.

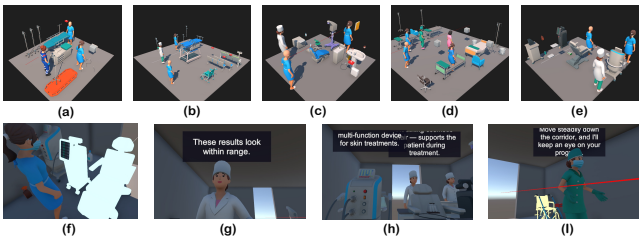


Figure 3: Case study spanning five generated scenarios and four representative interactions. Top row: environment perspectives of the placed scene for (a) ED Triage, (b) Therapy Gym, (c) Dental, (d) Ward, and (e) Cosmetic. Bottom row: runtime interactions captured during user play sessions, where (f) the nurse leads the user to a highlighted chair, (g) the user discusses results with the doctor, (h) the instrument displays its usage information, and (i) the nursing assistant directs the user to a highlighted wheelchair.

5 Discussion and Conclusion

We presented an LLM-based multi-module agent that authors interactive immersive healthcare scenes from a single natural-language description, lowering the barrier from code and 3D tooling to natural language commands. The closed-loop architecture separates a heavy planning frontier model from light, deterministically bounded VLM critics, and grounds reflection in three orthogonal screenshots instead of a heavyweight 3D scene encoder.

Limitations. Several limitations should be noted. First, the quality of generated experiences depends on the capabilities of the underlying LLM, which may produce inaccurate or inconsistent outputs. Second, the Operation layer currently targets Unity through a fixed action map, which constrains the types of interactions that can be generated to those exposed by our prefab and skill libraries. Lastly, while the Reflection module provides self-correction, it cannot guarantee medical accuracy in all cases, and the broader ethical, legal, and social implications of deploying virtual agents in care settings [12] require human expert review before any clinical use. Finally, a full ablation over architectural choices and frontier-model alternatives, together with multi-judge LLM-as-judge evaluation, is left to an extended version of this work.

Future Work. Our evaluation relies on automated metrics; the most important next step is an expert-review and user study in which students and domain experts play the generated scenes and rate their quality, pacing, and learning value. In addition, future directions include broader clinical-domain coverage, integrating real-time physiological signals (e.g., heart rate, galvanic skin response) for context-aware, and empathic adaptation [5].

Acknowledgments

This work was supported by NSF Awards No. 2420351 & 2420352.

REFERENCES

[1] V. Chheang, S. Sharmin, R. Márquez-Hernández, M. Patel, D. Rajasekaran, G. Caulfield, B. Kiafar, J. Li, P. Kullu, and R. L. Barmaki. Towards anatomy education with generative ai-based virtual assistants in immersive virtual reality environments. In *2024 IEEE international conference on artificial intelligence and eXtended and virtual reality (AIxVR)*, pp. 21–30. IEEE, 2024.

[2] B. G. Collaco, S. A. Haider, S. Prabha, C. A. Gomez-Cabello, A. Genovese, N. G. Wood, S. P. Bagaria, N. Gopala, C. Tao, and A. J. Forte. The role of agentic artificial intelligence in healthcare: a scoping review. *npj Digital Medicine*, 2026.

[3] W. Feng, W. Zhu, T.-j. Fu, V. Jampani, A. Akula, X. He, S. Basu, X. E. Wang, and W. Y. Wang. Layoutgpt: Compositional visual planning and generation with large language models. *Advances in neural information processing systems*, 36:18225–18250, 2023.

[4] D. Freeman, S. Reeve, A. Robinson, A. Ehlers, D. Clark, B. Spanlang, and M. Slater. Virtual reality in the assessment, understanding, and treatment of mental health disorders. *Psychological medicine*, 47(14):2393–2400, 2017.

[5] K. Gupta, Y. Zhang, T. S. Gunasekaran, N. Krishna, Y. S. Pai, and M. Billinghurst. Caevr: Biosignals-driven context-aware empathy in virtual reality. *IEEE Transactions on Visualization and Computer Graphics*, 30(5):2671–2681, 2024.

[6] Y. He, A. Li, B. Liu, Z. Yao, and Y. He. Medorch: Medical diagnosis with tool-augmented reasoning agents for flexible extensibility. *arXiv preprint arXiv:2506.00235*, 2025.

[7] H. G. Hoffman, G. T. Chambers, W. J. Meyer III, L. L. Arceneaux, W. J. Russell, E. J. Seibel, T. L. Richards, S. R. Sharar, and D. R. Patterson. Virtual reality as an adjunctive non-pharmacologic analgesic for acute burn pain during medical procedures. *Annals of behavioral medicine*, 41(2):183–191, 2011.

[8] Z. Hu, A. Iscen, A. Jain, T. Kipf, Y. Yue, D. A. Ross, C. Schmid, and A. Fathi. Scenecraft: An llm agent for synthesizing 3d scenes as blender code. In *ICML*, pp. 19252–19282, 2024.

[9] J. Li, Y. Lai, W. Li, J. Ren, M. Zhang, X. Kang, S. Wang, P. Li, Y.-Q. Zhang, W. Ma, et al. Agent hospital: A simulacrum of hospital with evolvable medical agents. *arXiv preprint arXiv:2405.02957*, 2024.

[10] Y. Liu, X. Cao, T. Chen, Y. Jiang, J. You, M. Wu, X. Wang, M. Feng, Y. Jin, and J. Chen. From screens to scenes: A survey of embodied ai in healthcare. *Information Fusion*, 119:103033, 2025.

[11] J. S. Park, J. O'Brien, C. J. Cai, M. R. Morris, P. Liang, and M. S. Bernstein. Generative agents: Interactive simulacra of human behavior. In *Proceedings of the 36th annual acm symposium on user interface software and technology*, pp. 1–22, 2023.

[12] C. Rudschies and I. Schneider. Ethical, legal, and social implications (elsi) of virtual agents and virtual reality in healthcare. *Social Science & Medicine*, 340:116483, 2024.

[13] T. Song, F. Pabst, U. Eck, and N. Navab. Enhancing patient acceptance of robotic ultrasound through conversational virtual agent and immersive visualizations. *IEEE Transactions on Visualization and Computer Graphics*, 2025.

[14] T. Sumers, S. Yao, K. R. Narasimhan, and T. L. Griffiths. Cognitive architectures for language agents. *Transactions on Machine Learning Research*, 2023.

[15] F.-Y. Sun, W. Liu, S. Gu, D. Lim, G. Bhat, F. Tombari, M. Li, N. Haber, and J. Wu. Layoutvlm: Differentiable optimization of 3d layout via vision-language models. In *2025 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pp. 29469–29478. IEEE, 2025.

[16] W. Tan, W. Zhang, X. Xu, H. Xia, Z. Ding, B. Li, B. Zhou, J. Yue, J. Jiang, Y. Li, et al. Cradle: Empowering foundation agents towards general computer control. *arXiv preprint arXiv:2403.03186*, 2024.

[17] J. C. Uhl, H. Schrom-Feiertag, G. Regal, K. Gallhuber, and M. Tscheligi. Tangible immersive trauma simulation: is mixed reality the next level of medical skills training? In *Proceedings of the 2023 chi conference on human factors in computing systems*, pp. 1–17, 2023.

[18] C. Wang, H. Zhong, M. Chai, M. He, D. Chen, and J. Liao. Chat2layout: Interactive 3d furniture layout with a multimodal llm. *IEEE transactions on visualization and computer graphics*, 2025.

[19] G. Wang, Y. Xie, Y. Jiang, A. Mandlekar, C. Xiao, Y. Zhu, L. Fan, and A. Anandkumar. Voyager: An open-ended embodied agent with large language models. *Transactions on Machine Learning Research*, 2024.

[20] Y. Wang, Y. Pan, Z. Su, Y. Deng, Q. Zhao, L. Du, T. H. Luan, J. Kang, and D. Niyato. Large model based agents: State-of-the-art, cooperation paradigms, security and privacy, and future trends. *IEEE Communications Surveys & Tutorials*, 2025.

[21] Z. Wang, J. Wu, L. Cai, C. H. Low, X. Yang, Q. Li, and Y. Jin. Medagent-pro: Towards evidence-based multi-modal medical diagnosis via reasoning agentic workflow. *arXiv preprint arXiv:2503.18968*, 2025.

[22] Z. Xi, W. Chen, X. Guo, W. He, Y. Ding, B. Hong, M. Zhang, J. Wang, S. Jin, E. Zhou, et al. The rise and potential of large language model based agents: A survey. *Science China Information Sciences*, 68(2):121101, 2025.

[23] Y. Yang, F.-Y. Sun, L. Weihs, E. VanderBilt, A. Herrasti, W. Han, J. Wu, N. Haber, R. Krishna, L. Liu, et al. Holodeck: Language guided generation of 3d embodied ai environments. In *2024 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pp. 16277–16287. IEEE, 2024.

[24] X. T. Zhu, H. Cheerman, M. Cheng, S. R. Kiami, L. Chukoskie, and E. McGivney. Designing vr simulation system for clinical communication training with llms-based embodied conversational agents. In *Proceedings of the Extended Abstracts of the CHI Conference on Human Factors in Computing Systems*, pp. 1–9, 2025.